Function 15: Analyze dc:identifier Match with MMS ID

Overview

Function 15 analyzes bibliographic records to determine whether their dc:identifier fields contain a value that exactly matches the MMS ID. This function creates two separate CSV files: one for records where the MMS ID appears as a dc:identifier, and another for records where it doesn't, along with all their actual dc:identifier values.

What It Does

This function processes records (single or batch) and categorizes them based on dc:identifier matching:

- **Matching**: Records that HAVE a dc:identifier exactly matching their MMS ID
- **Non-matching**: Records that DO NOT have a dc:identifier matching their MMS ID

Output Files

A timestamped directory is created in your Downloads folder containing up to three CSV files:

Directory: `~/Downloads/CABB_identifier_analysis_YYYYMMDD_HHMMSS/`

1. **identifier_matching_TIMESTAMP.csv**
   - Columns: `MMS ID`, `dc:identifier`
   - Contains records where MMS ID appears in dc:identifier fields
   - Single row per matching record
2. **identifier_non_matching_TIMESTAMP.csv**
   - Columns: `MMS ID`, `dc:identifier_1`, `dc:identifier_2`, `dc:identifier_3`, etc.
   - Contains records without MMS ID match
   - Shows ALL dc:identifier values the record actually has
   - Number of columns adapts to maximum identifiers found
3. **identifier_failed_TIMESTAMP.csv**
   - Columns: `MMS ID`, `Error`
   - Contains records that failed to process
   - Shows specific error message for each failure
   - Only created if there are failed records

Note: The output directory path is automatically copied to the "Set ID" field for your reference.

Key Features

- **Single or batch processing**: Works with one MMS ID or a full set
- **Exact matching**: Only matches if dc:identifier exactly equals MMS ID
- **Complete inventory**: Non-matching CSV shows all existing identifiers
- **Batch processing**: Efficient API calls (100 records per batch)

- **Progress tracking**: Real-time progress bar for batch operations
- **Kill switch**: Stop processing if needed
- **Automatic naming**: Timestamped filenames prevent overwrites
- **UTF-8 support**: Preserves all character encodings

The Need for This Function

Identifier Quality Assurance

In Alma Digital, the MMS ID is the fundamental identifier for bibliographic records. Some workflows or migration processes may add the MMS ID as a dc:identifier field, while others may not. This function helps answer:

- **Which records have MMS ID as dc:identifier?**
  - Important for certain integrations
  - May be required for specific workflows
  - Could affect discovery or linking systems
- **Which records are missing MMS ID in dc:identifier?**
  - Identify records that need updating
  - Understand identifier patterns across collections
  - Track consistency in metadata practices
- **What identifiers DO these records have?**
  - See alternative identifiers in use
  - Understand identifier schemes
  - Identify potential conflicts or duplicates

Use Cases

Quality Control:

- Verify MMS ID appears in dc:identifier when required
- Check for consistent identifier practices
- Identify records needing correction

Migration Analysis:

- Assess how MMS IDs were handled during migration
- Compare identifier schemes across batches
- Track identifier standardization progress

Troubleshooting:

- Debug linking issues related to identifiers
- Investigate discovery problems
- Resolve duplicate identifier conflicts

Collection Analysis:

- Understand identifier patterns by collection
- Compare different sets or batches
- Assess metadata quality across collections

How It Works

Step-by-Step Process

1. **Input**:
    - **Single Mode**: Enter MMS ID in the "Single Record" field
    - **Batch Mode**: Load a set using "Load Set Members" or CSV file
2. **Execute Function**:
    - Click Function 15 from dropdown
    - Progress bar appears (for batch processing)
    - Status updates show progress
3. **Batch Fetching** (per 100 records):
    - Send batch GET request to Alma Bibs API
    - Receive XML for up to 100 records at once
    - Parse each record's Dublin Core section
4. **Identifier Extraction** (per record):
    - Extract all `dc:identifier` elements
    - Check if MMS ID appears in the list
    - Categorize as matching or non-matching
5. **CSV Creation**:
    - **Matching file**: Write simple 2-column CSV
    - **Non-matching file**: Write multi-column CSV with all identifiers
    - **Failed file**: Write 2-column CSV with MMS ID and error message (if any failures)
    - Column count adapts to maximum identifiers found
6. **Completion**:
    - Display success message with statistics
    - Output directory created in Downloads folder
    - Directory path copied to "Set ID" field
    - Log shows directory location and record counts

Matching Logic

Python Implementation:

```
# Extract all dc:identifier values
identifiers = self._extract_dc_field("identifier", "dc")
```

```
# Check if MMS ID is in the identifier list
if mms_id in identifiers:
    # MATCH: Write to matching CSV
    matching_rows.append({
        "MMS ID": mms_id,
        "dc:identifier": mms_id
    })
else:
    # NO MATCH: Write to non-matching CSV with all identifiers
    row = {"MMS ID": mms_id}
    for idx, identifier in enumerate(identifiers, start=1):
        row[f"dc:identifier_{idx}"] = identifier
    non_matching_rows.append(row)
```

Example Scenarios

Scenario 1: Record WITH MMS ID match

- MMS ID: `991234567890123`
- dc:identifier fields:
  - `991234567890123` ✓
  - `Grinnell:12345`
  - `http://hdl.handle.net/11084/5678`
- **Result**: Appears in `identifier_matching_*.csv`

Scenario 2: Record WITHOUT MMS ID match

- MMS ID: `991234567890123`
- dc:identifier fields:
  - `Grinnell:12345`
  - `dg_12345`
  - `http://hdl.handle.net/11084/5678`
- **Result**: Appears in `identifier_non_matching_*.csv` with all three identifiers shown

Scenario 3: Record with NO identifiers

- MMS ID: `991234567890123`
- dc:identifier fields: (none)
- **Result**: Appears in `identifier_non_matching_*.csv` with empty identifier columns

Output Examples

Matching CSV (identifier_matching_20260305_143022.csv)

```
MMS ID,dc:identifier
991234567890123,991234567890123
991234567890456,991234567890456
991234567890789,991234567890789
```

Non-Matching CSV (identifier_non_matching_20260305_143022.csv)

```
MMS ID,dc:identifier_1,dc:identifier_2,dc:identifier_3
991234567891111,Grinnell:12345,dg_12345,http://hdl.handle.net/11084/5678
991234567892222,Grinnell:67890,http://hdl.handle.net/11084/9012,
991234567893333,dg_45678,,
991234567894444,,,
```

Note: The number of `dc:identifier_#` columns adapts to the maximum number of identifiers found across all non-matching records.

Failed CSV (identifier_failed_20260305_143022.csv)

```
MMS ID,Error
991234567895555,Record not returned in batch API call
991234567896666,Record not returned in batch API call
991234567897777,Network timeout during API call
```

Note: This file is only created if there are failed records. The Error column shows the specific reason each record failed to process.

Best Practices

When to Use This Function

Use Function 15 when you need to:

- Verify MMS ID appears in dc:identifier fields
- Audit identifier consistency across a collection
- Prepare for workflows that require MMS ID as identifier
- Troubleshoot linking or discovery issues
- Analyze identifier patterns in your records

Consider alternatives when you need:

- All identifier types categorized (use Function 8)
- Full Dublin Core metadata export (use Function 3)
- Only Handle URL validation (use Function 9)

Workflow Tips

1. **Start Small**: Test with a few records first
2. **Review All Files**: Check matching, non-matching, and failed CSVs in the output directory
3. **Note Statistics**: Log shows counts of matching vs. non-matching vs. failed
4. **Find Output**: Look in your Downloads folder for `CABB_identifier_analysis_*` directories
5. **Directory Path**: Automatically copied to "Set ID" field for easy reference

6. **Check Failures**: If there's a failed CSV, review error messages and consider re-running those MMS IDs
7. **Use Filters**: Excel/Sheets filters help analyze large result sets

Interpreting Results

High Match Rate (most records in matching CSV):

- Indicates MMS ID is consistently added as dc:identifier
- Suggests good identifier practices
- May indicate successful migration or batch update

High Non-Match Rate (most records in non-matching CSV):

- Indicates MMS ID typically NOT in dc:identifier
- Check if this is expected for your collections
- May indicate need for batch identifier update

Mixed Results:

- Compare patterns between matched and non-matched groups
- Look for collection-specific or date-specific patterns
- Consider batch updates for consistency

Common Issues

Problem: Function runs but creates empty CSVs

- **Cause**: No records loaded or API connection issue
- **Solution**: Verify set is loaded and API is connected

Problem: All records appear in non-matching CSV

- **Cause**: MMS IDs are not being added as dc:identifier
- **Solution**: This may be expected; review your metadata standards

Problem: Many records in failed CSV

- **Cause**: API issues, network problems, or invalid MMS IDs
- **Solution**: Check the Error column in identifier_failed_*.csv for specific reasons
- **Tip**: Re-run just the failed MMS IDs after resolving issues

Problem: Non-matching CSV has many columns

- **Cause**: Some records have many dc:identifier values
- **Solution**: Normal behavior; use horizontal scrolling in Excel/Sheets

Technical Details

API Efficiency

- **Batch calls**: Fetches up to 100 records per API call

- **Example**: 1,000 records = 10 API calls (vs. 1,000 individual calls)
- **Performance**: Significant speedup for large sets
- **Rate limits**: Respects Alma API rate limits

File Naming Convention

- **Directory Format**: `~/Downloads/CABB_identifier_analysis_YYYYMMDD_HHMMSS/`
- **CSV Format**: `identifier_(matching|non_matching|failed)_YYYYMMDD_HHMMSS.csv`
- **Example Directory**: `~/Downloads/CABB_identifier_analysis_20260305_143022/`
- **Example Files**:
  - `identifier_matching_20260305_143022.csv`
  - `identifier_non_matching_20260305_143022.csv`
  - `identifier_failed_20260305_143022.csv` (only if failures occurred)
- **Benefit**: Timestamps prevent accidental overwrites; organized in separate folder per analysis

Character Encoding

- **Format**: UTF-8 with BOM
- **Compatibility**: Opens correctly in Excel, Google Sheets, and text editors
- **Preserves**: Special characters, diacritics, and Unicode symbols

Related Functions

- **Function 1**: Fetch and view single record XML (inspect dc:identifier fields)
- **Function 3**: Export full Dublin Core metadata to CSV (includes all identifiers)
- **Function 8**: Export categorized identifier types (dg_, *Grinnell:*, Handle)
- **Function 7**: Add Grinnell: dc:identifier fields (batch update identifiers)

Version History

- **v1.0** (March 2026): Initial implementation
  - Single and batch processing
  - Two-file output system
  - Dynamic column adaptation for non-matching file
  - Batch API efficiency

Function 15 is part of the CABB (Crunch Alma Bibs in Bulk) application suite for Alma Digital metadata management.